



LAB 05:

Sprintf, Object Pooling, and DMA

Provided Files

- main.c
- gba.c, gba.h
- game.c, game.h
- font.c, font.h
- Example.gba

Files to Edit/Add

- .vscode
 - settings.json
 - tasks.json
 - main.c
 - gba.c
 - game.c, game.h
-

Instructions

In this lab you will be completing several different TODOs, which will complete a simple Brick Breaker game. Each TODO represents a component of the game, and is broken down into sub-TODOs. Your code will likely not compile until you complete an **entire TODO block**, at which point the game should compile with the new component added and working.

After you download and unzip the files, add your .vscode/tasks.json, and then **compile and run** it. At this point, you should see the start screen. Complete the TODOs in order, paying close attention to the instructions.

TODO 1: Score

We have provided drawChar() and drawString() for you. We want to be able to see our current progress to victory during the game state.

TODO 1.0

- In goToGame() within main.c, draw the word "score: " at (180, 1) using the color CYAN.



- **Note:** We draw "score:" in `goToGame()`, but the actual number in `game()` because the "score:" text does not change, so we only need to draw it once. The actual score, however, can change at any time, so we have to account for that possibility every frame.

TODO 1.1

- In `game()`, use `printf()` to save the current score (from variable `score`) in the text form to a character array.
 - The char array has already been created for you, called `scoreBuffer`. `printf()` is a function in `stdio.h`, which we have `#include'd` for you. It returns `null` and sets the contents of a supplied address to be the following formatterString:
 - `printf(ptrName, formatterString, variables, ...)`
 - This syntax is exactly like `printf()`, but with an extra argument at the beginning (the `ptrName` to save to).

TODO 1.2

- We want to erase the previous score before we draw the new one! Draw an `OFFWHITE` rectangle at (220, 1) that is the size of 2 characters (since we want to be able to reach double digit scores).
 - **Hint:** Look back at `font.c` – how wide and how tall is each character?
- Then, draw the score at this location (using `drawString()` with `scoreBuffer` as the character array parameter input) in `CYAN`.

Build and run. You should be able to travel to the game state and see the current score. Right now, it will be 0. In the next TODO, we will add blocks that can be broken to increase the score.

TODO 2: Blocks

Now we will add the bricks to be broken! This will take place in `game.h` and `game.c`.

TODO 2.0

- In `game.h`, complete the struct for the blocks, typedef-ed as `BLOCK`. `BLOCK` should have the following members: `x`, `y`, `oldX`, `oldY`, `xVelocity`, `width`, `height`, `color`, `active`, and `needToErase`.
 - All of the `BLOCK` members should be of type `int` except `color`, which should be an unsigned `short`.



TODO 2.1

- Also in `game.h`, declare an array of `BLOCK` structs named `blocks` that is of size `BLOCKCOUNT`.
 - `BLOCKCOUNT` is a macro already defined for you.
- Initialize `blocks` in `game.c`.

TODO 2.2

- In `game.c`, uncomment the first for loop in `initBlocks()`. In this loop, set the `active` property of the current block to 1, indicating that it is active. Then uncomment the rest of this function, which initializes all the blocks and their members. Review this logic so you understand how to initialize your own pool of objects.

TODO 2.3

- Uncomment the `updateBlocks()` function. Add a conditional inside of the if statement to perform collision checks if the block is *active*.
 - Inside of the main collision check, set the block to inactive and needed to erase (`active = 0`, `needToErase = 1`), and increment score by 1
 - This function loops through the `blocks` array. If the block is *inactive*, the function should do nothing. If it is *active*, it should perform the collision checks we wrote for you.

TODO 2.4

- In `updateGame()`, call `updateBlocks()` function.

TODO 2.5

- Also in `game.c`, complete `drawBlocks()`. This should be looping through the `blocks` array, and for each block, it'll be checking the `active` and `needToErase` members.
 - If the block is active, then it should erase the old block at its old location with `OFFWHITE` and draw the block at its current location with the block's color.
 - Else, if the block is not active and the block needs to be erased, then erase the block at its current location with `OFFWHITE` and set the block's `needToErase` to 0.
 - **Note:** Drawing and erasing the blocks should be calling the `drawRectangle()` function using the correct members of the block struct as the function's arguments.



TODO 2.6

- In `drawGame()`, call `drawBlocks()` function.

TODO 2.7

- Complete `activateBlock()`. This should iterate through all the blocks within the blocks array to find the first inactive block in the pool and initialize it. Initialize the block like so:
 - `active: 1`
 - `needToErase: 0`

When you are finished initializing, break out of the loop.

Compile and run. When you enter the game state, you should see layers of blocks of different colors. **Note: When you hit a block, sometimes it will seem to not disappear. This is because we are calling `newBlock` to immediately set a block to be active again which is sometimes the block we just destroyed. This is intended, don't worry!**

TODO 3: DMA

For our game to be fast, we need to use DMA. The simplest way to do this is to write a function that sets up the registers of the specified channel.

TODO 3.0

- In `gba.c`, complete the `DMANow` function.
 - This function should set up all the registers of the given DMA channel and turn it on for us. This allows us to use DMA whenever we need it with a single line, without having to set all the registers line-by-line in every location we want to use it.
 - There are additional comments in the `DMANow` function that will help you write it.

TODO 3.1

- Delete or comment out the code in the `fillScreen()` function, and rewrite the `fillScreen()` function to use DMA, using your new `DMANow` function.
- Make sure to use **DMA channel 3**. You may not use any loops.
 - **Hint:** If you are copying one thing (one single source) to every pixel in the `videoBuffer`, what do you need to tell the DMA control to do (or not do) to the source? Look at `gba.h` for helper macros!
 - **Hint:** Make sure the `src` and `dest` parameters are *addresses* to the locations you are copying from and to.



TODO 3.2

- Rewrite the drawRectangle function to use DMA, using your new DMANow function.
- Make sure to use **DMA channel 3**. You may use only one loop.
 - **Hint:** You must use one loop here because, unlike fillScreen, the area you are copying to (dest) is not contiguous. Each row of the rectangle is, though. Use DMA to draw one row at a time.
 - **Hint:** Make sure the src and dest parameters are *addresses* to the locations you are copying from and to.

Build and run. The game should look the same, but a lot snappier during transitions.

You will know your lab runs correctly if:

- On compilation, you see “GAME BOY” in rainbow letters printed to the screen, with “Spring 2026” written underneath in red.
- In the game state, you can see the score written in cyan next to “score:”
- The blocks in the game state perform equivalent to Example.gba with a minimal amount of flicker and lag.